Since 1983

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Department of Computer Engineering and Technology

MINI PROJECT REPORT

Computer Networks Laboratory

(CSE20110)

Laboratory Continuous Assessment 3

Nethryx - Network Packet Analyzer Dashboard

Submitted by

Sr. No.	Student Name	PRN
1	Krish Bhensdadia	1262250028
2	Pooja Shah	1262251181
3	Aayush Potdar	1262250835
4	Adarsh Bhosale	1262252198
5	Aditya Dharankar	1262250024

Group No. **5** Panel - **E** Batch - **E1**

Third Year B.Tech. Computer Engineering and Technology

Under the guidance of

Dr. Swati

Academic Year 2026–2027

Submission Date - **21st October 2026**

Since 1983

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Department of Computer Engineering and Technology

CERTIFICATE

This is to certify that the mini-project entitled "Nethryx - Network Packet Analyzer Dashboard" has been carried out by the following students of Third Year B.Tech. Computer Engineering and Technology, Panel E, Batch E1, Group No. 5, under my guidance during the academic year 2026–2027.

Student Name	**PRN**
Krish Bhensdadia	1262250028
Pooja Shah	1262251181
Aayush Potdar	1262250835
Adarsh Bhosale	1262252198
Aditya Dharankar	1262250024

The report is submitted towards the mini-project component of Laboratory Continuous Assessment 3 for the Computer Networks Laboratory course.

Faculty Signature _____

Dr. Swati

Project Guide and Course Faculty

Date _____

Place: Pune

Acknowledgement

We express our sincere gratitude to our project guide, Dr. Swati, for the guidance provided for this Computer Networks Laboratory mini-project. We also thank the Department of Computer Engineering and Technology, MIT World Peace University, Pune, for providing the academic setting in which we could develop and document Nethryx. We acknowledge the open-source documentation and tools used in the implementation, including FastAPI, Scapy, React, Playwright and the Groq API. The team remains responsible for validating the application and the results reported in this document.

Group No. 5

Abstract

Nethryx is a web-based Network Packet Analyzer Dashboard designed to make packet-capture analysis and browser-generated network activity easier to inspect. It accepts PCAP and PCAPNG files and uses Scapy to extract packet counts, transferred bytes, protocol classifications, timestamps, IP endpoints and DNS-derived host names. A React dashboard presents protocol distributions, traffic over time, host summaries and rule-based indicators for patterns such as SYN requests to many distinct ports and unusually long DNS names. The project also includes an AI-assisted browser automation workflow: a Groq language model converts a natural-language task into a bounded set of steps, while Playwright executes supported actions on selected websites. When enabled and properly configured, live packet capture can run alongside automation and display packet events and browser network requests. The backend is implemented with FastAPI and exposes REST endpoints and a WebSocket channel for automation updates. The system is intended for educational inspection of captures, not as a replacement for a full network intrusion detection system. Protocol labels are heuristic, TLS payloads remain encrypted, and flagged patterns require human interpretation. Runtime measurements and screenshots should be recorded during the final demonstration rather than inferred from source code.

Keywords: Network packet analysis, PCAP, Scapy, FastAPI, browser automation, traffic monitoring

Table of Contents

1 Introduction and Project Objectives 6
2 Proposed Design and Requirements 8
3 Implementation and Technical Working 10
4 Testing and Results 12
5 Discussion and Conclusion 14
References 16
Appendices 17

List of Figures

Figure 2.1 Nethryx system architecture 8
Figure 3.1 PCAP dashboard screenshot (add during final demo) 11
Figure 4.1 Test execution evidence (add during final demo) 13

List of Tables

Table 1.1 Project objectives and verification methods 7
Table 2.1 Tools and system requirements 8
Table 2.2 Network addressing and protocol selection 9
Table 4.1 Test plan and status 12
Table 4.2 Objective achievement 13

1 Introduction and Project Objectives

1.1 Introduction and Background

Network packet analysis examines captured frames and packets to understand communication endpoints, protocols, traffic volume and timing. PCAP and PCAPNG files preserve packet records for offline inspection, while tools such as Scapy provide programmatic access to packet layers and fields [1]. Nethryx applies these concepts through a web dashboard so that a student can upload a capture and review summarized information without manually inspecting every packet.

The application also explores live traffic observation during browser automation. FastAPI provides HTTP endpoints and a WebSocket channel for streaming task events [2]. Playwright drives a browser through controlled actions and exposes browser network-request metadata [3]. Groq's API is used to plan supported browser steps and to answer capture questions from extracted summary statistics; the model is instructed to distinguish observations from possible interpretations. These components are combined for a laboratory learning workflow, with explicit limitations around encrypted traffic and heuristic classification.

1.2 Problem Statement

Manual packet inspection can be time-consuming for beginners because a capture may contain many packets and several protocol types. A user may need to identify the busiest protocols, estimate data volume, observe traffic over time, locate visible hosts and notice patterns that merit further inspection. A separate challenge is correlating browser actions with network activity during a demonstration. Nethryx addresses these needs with a summarized dashboard, a rule-based pattern panel and an optional browser automation/live-capture workflow.

1.3 Scope

The intended users are students and instructors studying computer networks. The offline analyzer supports uploaded .pcap and .pcapng captures up to 200 MB, with at most 300,000 packet rows retained for detailed charts and filtering; total packet and byte counts are tracked across the file. The live workflow captures traffic from the selected network adapter and may include traffic from other applications. Browser automation is restricted to configured allowed domains (YouTube, GitHub, Google, Wikipedia and LinkedIn by default). The application does not decrypt TLS, prove malicious intent, or provide a complete IDS/IPS.

The current backend keeps uploaded-capture analysis in process memory, so persistence across restarts is not provided.

1.4 Objectives

The project objectives are to parse supported capture files, summarize traffic in a visual dashboard, identify selected patterns using transparent rules, and demonstrate browser task execution with optional live packet observation.

Objective ID	Objective	Verification Method
O1	Parse valid PCAP/PCAPNG files and reject unsupported/invalid uploads.	Upload valid, wrong-extension and invalid-header files.
O2	Summarize packet volume, protocols, timeline and visible hosts.	Compare dashboard values with capture/parser output.
O3	Show transparent pattern flags and optional browser automation/live events.	Use a known test capture and run a permitted browser task.

2 Proposed Design and Requirements

2.1 Architecture and Network Topology

The user interacts with a React single-page interface. The frontend sends capture uploads and summary requests to the FastAPI backend over HTTP and connects to a WebSocket endpoint for automation events. Scapy parses uploaded captures and extracts packet-level fields. For live operation, Playwright controls Chrome while the optional Scapy sniffer records packets through the local adapter (Npcap and elevated privileges may be required on Windows). Groq is called over HTTPS for task planning and capture-question responses. Figure 2.1 summarizes these modules and their main data paths.

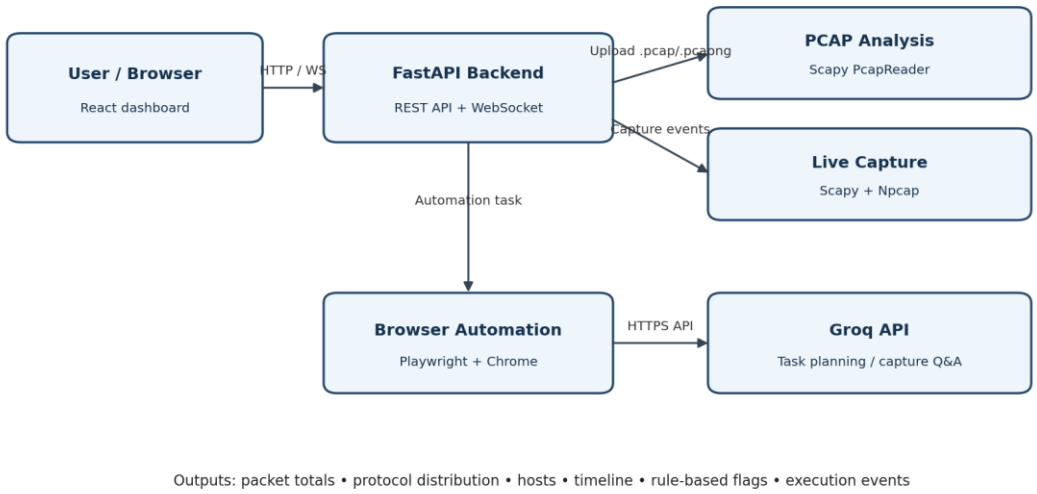


Figure 2.1 Nethryx system architecture

2.2 Tools and System Requirements

The source repository defines a React 18 frontend built with Vite, Tailwind CSS, Recharts and Lucide icons. The backend uses Python, FastAPI, Uvicorn, Scapy, HTTPX, python-dotenv and Playwright. The README specifies Python 3.10+ and Node.js 18+. Exact installed versions and hardware specifications were not included in the supplied project files and should be recorded from the demonstration machine.

Tool / Resource	Version / Specification	Purpose
Python / FastAPI	Python 3.10+; versions unpinned	Backend API

Tool / Resource	Version / Specification	Purpose
Scapy	Version unpinned	PCAP parsing / live capture
React / Vite	React 18; Vite 5.x	Frontend dashboard
Recharts / Tailwind	Recharts 2.x; Tailwind 3.x	Charts / styling
Playwright / Chrome	Browser install required	Browser automation
Groq API	Model configurable in .env	AI planning / capture Q&A
Npcap (Windows)	Required for live capture	Adapter packet access

2.3 Addressing and Protocol Selection

Nethryx analyzes captures rather than configuring a router or assigning static addresses. The following values describe the local development setup shown in the project configuration; deployed host addresses depend on the machine and hosting environment.

Device / Interface	IP / Prefix	Gateway	Role
Development PC	DHCP; varies	Network default gateway	Runs UI and browser
Backend API	127.0.0.1:8000	N/A (local)	REST / WebSocket
Frontend server	localhost:5173	N/A (local)	React UI
Network adapter	Record actual IP	Record if required	Optional live capture

HTTP is used for capture upload and analysis APIs; WebSocket is used for interactive automation status and packet events. The backend accepts only filenames ending in .pcap or .pcapng and checks known capture-file magic bytes before parsing. Protocol labels are derived from packet layers and, for HTTP/TLS, port-based heuristics; therefore a “TLS” label is not a decryption result. The backend avoids reporting incoming/outgoing totals because a raw capture does not reliably establish which endpoint is local.

3 Implementation and Technical Working

3.1 Methodology

The implementation is organized into three user-facing routes: a home page describing the tool, a PCAP upload and analysis dashboard, and an AI automation dashboard. For offline analysis, the user selects a capture file, the frontend posts it to /api/captures, and the backend validates and parses it. Summary statistics are returned as JSON and visualized using charts and host/protocol panels. For automation, the user submits a natural-language task; the backend requests a structured plan from Groq, validates the steps against allowed sites and action types, then runs browser actions with Playwright. Optional live capture emits packet statistics and rows while the task runs.

3.2 Configuration or Code Implementation

The principal backend modules are backend/main.py, backend/automation.py and backend/capture.py. main.py validates uploads, reads packet records with Scapy PcapReader, classifies packets, aggregates protocol and timeline statistics, maps DNS responses to host names, and flags two limited patterns: SYN traffic to 20 or more distinct destination ports for a source/destination pair, and DNS query names longer than 50 characters. It exposes capture listing, summary, packet filtering and an AI question endpoint. automation.py plans and validates browser steps, runs Playwright and streams events over WebSocket. capture.py performs optional live sniffing and writes a capture file. The frontend components App.jsx, PcapPage.jsx and Automation.jsx render the navigation, charts, automation timeline, browser screenshots and packet tables. The Groq key is read from the server environment; credentials must not be committed to source control.

3.3 Protocols and Technical Working

The browser communicates with FastAPI using HTTP requests for file upload and REST resources. During an automation run, a WebSocket carries plan, step, status, screenshot, packet and completion messages. Scapy reads packet layers such as IPv4/IPv6, TCP, UDP, ICMP and DNS. Packets are grouped by heuristic protocol label and integer-second timestamp for the dashboard. Browser request metadata is obtained from Chrome through Playwright and is separate from the raw packet capture. If the Groq key is missing, AI-dependent features return an error; if the capture file is invalid or cannot be parsed, the upload endpoint returns an error response.

3.4 Execution Procedure

1. Install Python 3.10 or later and Node.js 18 or later.
2. In backend/, create and activate a virtual environment, then run: pip install -r requirements.txt.
3. Copy .env.example to .env and configure GROQ_API_KEY on the backend only.
4. Start the API from backend/: python -m uvicorn main:app --reload --port 8000.
5. In a second terminal, open frontend/, run npm install and npm run dev, then visit http://localhost:5173.
6. Open the PCAP page and upload a valid .pcap or .pcapng file. Review packet totals, protocol charts, timeline, hosts and rule-based flags.
7. For automation, configure Groq and install the Playwright browser. On Windows, live packet capture may require Npcap and an Administrator terminal. Run only approved tasks on the configured allowed domains.

Insert a screenshot from the running application here before submission. The screenshot should show the Nethryx PCAP dashboard after a real capture has been analyzed, with the capture filename, packet total, protocol chart and traffic timeline visible. Do not use a mock-up or report unobserved values.

Figure 3.1 PCAP dashboard screenshot — add actual screenshot before submission

4 Testing and Results

4.1 Test Plan

Testing should be performed on the target demonstration machine with the backend and frontend running. The supplied source confirms that validation and analysis paths are implemented, but it does not include an executed test log or a sample capture with measured results. Accordingly, the statuses below distinguish checks that are defined by the code from runtime outcomes that still need to be recorded during demonstration.

Test ID	Test and Input	Expected Result	Observed Result	Status
T1	Upload valid .pcap or .pcapng file.	Capture parsed; totals and charts returned.	Implementation path exists; runtime evidence not supplied.	Pending runtime test
T2	Upload unsupported extension or invalid capture header.	Request rejected with an explanatory error.	Extension and magic-byte checks are implemented in main.py.	Code-inspected; runtime test pending
T3	Run an allowed browser task with Groq configured; enable capture if prerequisites are available.	Plan and step events appear; packet events appear if capture starts.	Automation and live-capture event paths exist; runtime evidence not supplied.	Pending runtime test

4.2 Results and Execution Evidence

The supplied source code defines dashboard outputs for packet count, total bytes, protocol counts/bytes, timeline bins, public hosts and suspicious-pattern rules. No actual capture file, runtime log or screenshot was included in the supplied project archive, so numerical results cannot be reported responsibly here. Add the real dashboard screenshot and record the capture filename and values observed during the final demonstration.

Figure 4.1 Test execution evidence — add actual screenshot before submission

4.3 Objective Achievement

Objective achievement must be finalized using the test evidence collected during the demonstration. Static inspection shows implementation paths for upload parsing, dashboard summaries, rule-based flags and browser automation; successful runtime execution has not been independently evidenced by the supplied archive.

Objective ID	Evidence or Test ID	Outcome
O1	T1, T2	Implemented in source; runtime validation pending.
O2	T1	Dashboard aggregation and visualization paths implemented; measured output pending.
O3	T3	Automation and rule-based alert paths implemented; end-to-end test pending.

5 Discussion and Conclusion

5.1 Discussion and Comparative Analysis

Compared with reading packet rows manually, the dashboard is designed to provide an at-a-glance summary of packet volume, protocol distribution, traffic over time and visible hosts. Its advantage is presentation and workflow integration rather than a proven improvement in detection accuracy or processing speed. Wireshark remains more suitable for detailed packet dissection and protocol-specific investigation; Nethryx provides a lightweight educational overview and optional automation demonstration. No benchmark data was supplied, so no quantitative performance claim is made.

5.2 Advantages and Applications

Potential educational uses include understanding traffic composition, inspecting DNS and endpoint metadata, visualizing packet timing, and observing how browser actions generate network requests. The rule-based flags can help a learner select a capture region for further investigation. The automation workflow can make repeatable browsing tasks easier to demonstrate while showing corresponding network events.

5.3 Limitations

The protocol classifier uses packet-layer presence and port heuristics, so labels may not always match the application protocol. TLS payloads are encrypted and are not decoded. The two suspicious-pattern rules are simple indicators, not proof of an attack; benign traffic can trigger them and other threats will not be detected. Public-host summaries depend on IP classification and available DNS responses. Live capture observes the whole adapter, can include unrelated application traffic, and may require operating-system privileges. The current analysis state is held in memory, the detailed row limit is 300,000, and the archive contains no test captures or execution evidence. AI output depends on external Groq availability and should be checked against extracted statistics.

5.4 Challenges and Troubleshooting

The supplied repository documents several operational dependencies that should be checked during setup. The table records preventive troubleshooting guidance; replace or supplement it with the actual issues encountered and their verified fixes during the demonstration.

Issue	Cause or Diagnosis	Solution and Verification

Issue	Cause or Diagnosis	Solution and Verification
Backend unreachable	Frontend default API URL is localhost:8000; service may not be running.	Start Uvicorn on port 8000 and verify API connectivity.
AI planning fails	GROQ_API_KEY missing or API unavailable.	Set key in backend .env; never expose it in frontend or commit it.
Live capture unavailable	Npcap missing or terminal lacks privileges on Windows.	Install Npcap, run backend terminal as Administrator, and verify capture status.

5.5 Future Scope

Future work could add persistent storage for capture metadata, downloadable analysis reports, richer filters and conversation/session comparison, configurable detection rules, protocol-aware parsing, exportable CSV summaries, and automated unit/integration tests using known PCAP fixtures. The automation workflow could add explicit interface selection for live capture and better separation of browser-related traffic from other adapter traffic. Any advanced detection model should be evaluated against labeled captures before its alerts are treated as reliable.

5.6 Conclusion

Nethryx combines offline packet-capture parsing, visual traffic summaries, transparent rule-based indicators and an optional AI-assisted browser automation workflow in one web interface. The source implementation includes file validation, packet aggregation, REST endpoints, WebSocket events and frontend charts. The design is suitable for a Computer Networks Laboratory demonstration of packet-analysis concepts. Because the supplied archive does not contain actual test captures or runtime evidence, final validation, screenshots and observed test outcomes must be added after running the project on the demonstration machine.

References

References are limited to official documentation relevant to the libraries and services used in the implementation.

[1] Scapy Project, “Scapy Documentation.” https://scapy.readthedocs.io/ (accessed Oct. 20, 2026).

[2] FastAPI, “FastAPI Documentation.” https://fastapi.tiangolo.com/ (accessed Oct. 20, 2026).

[3] Microsoft, “Playwright Python Documentation.” https://playwright.dev/python/ (accessed Oct. 20, 2026).

[4] Groq, “Groq API Documentation.” https://console.groq.com/docs (accessed Oct. 20, 2026).

AI Usage Declaration

AI tools are part of the implemented application. The Groq API is used to convert a user task into structured browser steps and to answer questions using extracted capture statistics. Any AI-assisted report drafting or code assistance should be disclosed by the group according to course requirements. AI-generated interpretations must be checked against packet evidence; this report does not claim that runtime outputs were experimentally verified from the supplied archive.

Declaration: We take responsibility for the accuracy of this report and understand the submitted implementation and results. Sources and assistance used have been acknowledged.

Student Name	PRN	Signature
Krish Bhensdadia	1262250028	
Pooja Shah	1262251181	
Aayush Potdar	1262250835	
Adarsh Bhosale	1262252198	
Aditya Dharankar	1262250024	

Appendices

Appendix A Code and Configuration

The submitted project archive contains backend/main.py (capture parsing and REST API), backend/automation.py (Groq task planning, Playwright execution and WebSocket events), backend/capture.py (optional live capture), frontend/src/App.jsx, frontend/src/PcapPage.jsx, frontend/src/Automation.jsx and frontend/src/Glass.jsx. The archive should be submitted alongside this report or its repository URL should be included by the group. Do not include .env files or API keys.

Appendix B Additional Evidence

Attach the actual PCAP/PCAPNG file used for testing if permitted, a screenshot of the analyzed dashboard, a screenshot of the automation timeline and a brief test log. Record the capture source and ensure no sensitive traffic or personal data is shared.

Appendix C Individual Contributions

Complete this table according to the work actually performed by each team member. Do not assign contributions that were not completed.

Student Name and PRN	Work Completed	Evidence or Deliverable
Krish Bhensdadia / 1262250028	To be completed by the team based on actual contribution.	Specify files, tests or report sections.
Pooja Shah / 1262251181	To be completed by the team based on actual contribution.	Specify files, tests or report sections.
Aayush Potdar / 1262250835	To be completed by the team based on actual contribution.	Specify files, tests or report sections.
Adarsh Bhosale / 1262252198	To be completed by the team based on actual contribution.	Specify files, tests or report sections.
Aditya Dharankar / 1262250024	To be completed by the team based on actual contribution.	Specify files, tests or report sections.